

Préparation au Concours National Commun 2026 — Informatique MP

Épreuve d'Informatique

Filière MP — Durée : 4 heures

Algorithmique de la blockchain Ethereum : gaz, encodage, arbres de Merkle, aléa et programmation dynamique

Auteur : **Pr Agrégé El Hadiq Zouhair**

cpgeacademy.org — Tous droits réservés © cpgeacademy.org

Avis important. Ce document est une **initiative personnelle** du professeur, à but strictement pédagogique. Il ne s'agit **pas** d'un sujet officiel du Concours National Commun ; il s'en inspire uniquement par la forme et l'esprit. **Pour aller plus loin.** Les élèves peuvent traiter une **nouvelle version** de ce problème dotée d'un **autocorrecteur des réponses**, et suivre un **cours complet**, sur cpgeacademy.org.

Présentation et notations

Ethereum est un ordinateur mondial décentralisé. Ce problème étudie, de manière *largement indépendante* partie par partie, quelques structures et algorithmes qui le sous-tendent : la mesure des ressources (le *gaz*), l'encodage hexadécimal des adresses, les *arbres de Merkle*, la sélection aléatoire d'un validateur, et enfin un problème d'optimisation résolu par *programmation dynamique*.

Conventions.

- Le langage est Python 3. Les entiers Python sont de taille arbitraire.
- Pour une liste L , `len(L)` est sa longueur et $L[i]$ son terme d'indice i (à partir de 0).
- $[x]$ désigne la partie entière de x . $\log_b$ est le logarithme en base b .
- On rappelle que $n // b$ et $n \% b$ sont le quotient et le reste de la division euclidienne.

Les questions sont numérotées **en continu** sur tout le sujet.

ÉNONCÉ

Partie I — Unités et compteur de gaz

L'unité de compte d'Ethereum est l'*ether* ; sa plus petite subdivision est le *wei*, avec $1 \text{ ether} = 10^{18} \text{ wei}$. Chaque instruction exécutée consomme une quantité fixe de *gaz* ; une transaction fixe un plafond de gaz au-delà duquel l'exécution est interrompue.

Question 1. Écrire une fonction `ether_vers_wei(e)` qui, pour un entier $e \geq 0$, renvoie le nombre de wei correspondant à e ethers.

Question 2. Écrire `gaz_restant(plafond, couts)` où `couts` est la liste des coûts (entiers positifs) des instructions exécutées dans l'ordre. La fonction retranche les coûts un à un ; si, avant une instruction, le gaz disponible est *strictement* inférieur à son coût, l'exécution s'arrête et la fonction renvoie la chaîne "**panne de gaz**". Sinon elle renvoie le gaz restant.

Question 3. Un contrat exécute une boucle dont l'itération numéro i (pour $i \geq 0$) coûte $g/2^i$ unités de gaz, où g est un réel strictement positif. Montrer que le coût total $C = \sum_{i=0}^{+\infty} \frac{g}{2^i}$ est fini et

vaut exactement $2g$. En déduire que, quel que soit le nombre d'itérations effectivement exécutées, le coût cumulé est majoré par $2g$.

Partie II — Encodage hexadécimal des adresses

Une adresse Ethereum est un entier affiché en base 16 (hexadécimal), avec l'alphabet "0123456789abcdef". On étudie l'encodage d'un entier en base 16.

Question 4. Écrire `entier_vers_hex(n)` qui, pour un entier $n \geq 0$, renvoie la chaîne de ses chiffres hexadécimaux (sans le préfixe 0x), avec la convention `entier_vers_hex(0) == "0"`.

Question 5. Écrire `hex_vers_entier(s)` qui, pour une chaîne `s` de chiffres hexadécimaux minuscules, renvoie l'entier qu'elle représente.

Question 6. Pour $n \geq 1$, montrer que le nombre de tours de la boucle de `entier_vers_hex(n)` (donc le nombre de chiffres produits) est exactement $\lfloor \log_{16} n \rfloor + 1$. En déduire la complexité temporelle de `entier_vers_hex` en fonction du nombre de chiffres.

Question 7. Justifier la **terminaison** de la boucle de `entier_vers_hex` à l'aide d'un *variant de boucle*, puis prouver la **correction**, c'est-à-dire que pour tout $n \geq 0$, `hex_vers_entier(entier_vers_hex(n))` est égal à n .

Partie III — Arbres de Merkle

Un *arbre de Merkle* résume un ensemble de données par un seul haché, la *racine*. On se donne une fonction de combinaison

$$\text{parent}(a, b) = (31a + b) \bmod 1000003,$$

et les feuilles sont des entiers. La racine se calcule par niveaux : à chaque niveau, on remplace la liste courante par la liste des `parent` de couples consécutifs ; si la liste a une longueur impaire, on **duplique son dernier élément** avant de l'appairer. On s'arrête lorsqu'il ne reste qu'un élément.

Question 8. Écrire `racine_merkle(feuilles)` qui prend une liste non vide de feuilles et renvoie la racine. Vérifier à la main que `racine_merkle([1, 2, 3, 4])` vaut 1120.

Question 9. Soit n le nombre de feuilles. Montrer que le nombre total d'appels à `parent` effectués par `racine_merkle` est un $O(n)$. *Indication* : majorer la somme des tailles des niveaux successifs.

Question 10. On veut prouver l'*inclusion* d'une feuille sans révéler tout l'arbre. Écrire `preuve_inclusion(feuilles, i)` qui renvoie la liste des couples (frère, côté) rencontrés en remontant de la feuille d'indice i jusqu'à la racine, où le côté vaut 'L' ou 'R' et indique si le frère est à gauche ou à droite. Écrire ensuite `verifier(feuille, preuve, racine)` qui recalcule la racine à partir de la feuille et de la preuve, et renvoie un booléen.

Question 11. Montrer par récurrence sur la hauteur de l'arbre que, si la fonction `parent` est *résistante aux collisions* (on ne sait pas produire $(a, b) \neq (a', b')$ avec `parent(a, b) = parent(a', b')`), alors une preuve d'inclusion acceptée par `verifier` garantit que la feuille figure bien parmi les feuilles ayant produit la racine. Quelle est la taille d'une preuve d'inclusion en fonction de n ?

Partie IV — Sélection aléatoire d'un validateur

À chaque créneau (*slot*), un validateur donné est choisi comme proposeur avec probabilité $p \in]0, 1[$, indépendamment des autres créneaux. On note N le numéro (à partir de 1) du premier créneau où ce validateur est choisi.

Question 12. Donner, pour $k \geq 1$, la probabilité $\mathbb{P}(N = k)$. Reconnaître la loi de N et en déduire l'espérance $\mathbb{E}[N]$.

Question 13. Montrer que $\mathbb{P}(N > k) = (1 - p)^k$, puis que $\mathbb{P}(N < +\infty) = 1$ (*terminaison presque sûre* : le validateur est choisi en temps fini avec probabilité 1).

Question 14. Application numérique : pour $p = 1/4$, donner $\mathbb{E}[N]$ et la probabilité $\mathbb{P}(N \leq 4)$ sous forme de fraction irréductible.

Partie V — Partition équilibrée par programmation dynamique

On dispose d'une liste S d'entiers strictement positifs. Le *problème de partition* demande s'il existe un sous-ensemble de S dont la somme vaut exactement la moitié de la somme totale (les deux parts ont alors la même somme). C'est un problème classique lié aux preuves à divulgation nulle.

Question 15. Montrer que si la somme totale $T = \sum S$ est impaire, la réponse est nécessairement *non*. On suppose désormais T paire et on pose $C = T/2$.

Question 16. Le *problème de décision* associé (« existe-t-il un sous-ensemble de somme C ? ») est **NP**. Justifier en décrivant un *certificat* et l'algorithme de vérification, ainsi que sa complexité.

Question 17. On définit, pour $0 \leq i \leq \text{len}(S)$ et $0 \leq c \leq C$, le booléen $A[i][c] = \ll \text{il existe un sous-ensemble des } i \text{ premiers éléments de } S \text{ de somme } c \gg$. Donner la relation de récurrence vérifiée par $A[i][c]$ et son initialisation.

Question 18. En déduire une fonction `peut_partitionner(S)` utilisant un tableau à une dimension `dp` de taille $C + 1$. *Attention au sens de parcours* pour ne pas réutiliser deux fois le même élément. Donner sa complexité en temps et en espace, et expliquer le terme *pseudo-polynomial*.

Question 19. Prouver la correction de `peut_partitionner` en énonçant un *invariant* portant sur `dp` après le traitement des i premiers éléments, puis en le démontrant par récurrence sur i . Justifier enfin la terminaison.

CORRIGÉ

Partie I — Unités et compteur de gaz

Question 1. On multiplie par 10^{18} .

```
1 def ether_vers_wei(e):
2     return e * 10**18
```

Complexité $O(1)$ opérations sur grands entiers.

Question 2. On parcourt les coûts en maintenant le gaz disponible.

```
1 def gaz_restant(plafond, couts):
2     restant = plafond
3     for c in couts:
4         if restant < c:
5             return "panne de gaz"
6         restant -= c
7     return restant
```

La boucle est finie (`len(couts)` tours), d'où la terminaison ; complexité $O(m)$ avec $m = \text{len}(\text{couts})$.

Question 3. C'est une série géométrique de raison $1/2 \in]0, 1[$, donc convergente :

$$C = \sum_{i=0}^{+\infty} g \left(\frac{1}{2} \right)^i = \frac{g}{1 - \frac{1}{2}} = 2g.$$

Une exécution qui s'arrête après m itérations consomme $\sum_{i=0}^{m-1} g/2^i = g \frac{1 - (1/2)^m}{1 - 1/2} = 2g(1 - 2^{-m}) < 2g$. Le coût cumulé est donc *toujours* strictement majoré par $2g$, indépendamment de m .
□

Partie II — Encodage hexadécimal des adresses

Question 4.

```
1 def entier_vers_hex(n):
2     if n == 0:
3         return "0"
4     chiffres = "0123456789abcdef"
5     s = ""
6     while n > 0:
7         s = chiffres[n % 16] + s
8         n //= 16
9     return s
```

Question 5.

```
1 def hex_vers_entier(s):
2     chiffres = "0123456789abcdef"
3     n = 0
4     for c in s:
5         n = n * 16 + chiffres.index(c)
6     return n
```

Question 6. Soit $n \geq 1$ et d le nombre de chiffres produits. À chaque tour, n est remplacé par $\lfloor n/16 \rfloor$; la boucle s'arrête dès que n atteint 0. Le nombre de tours est donc le plus petit d tel que $\lfloor n/16^d \rfloor = 0$, c'est-à-dire $16^d > n$, soit $d > \log_{16} n$. Le plus petit tel entier est $d = \lfloor \log_{16} n \rfloor + 1$. Chaque tour coûte $O(1)$ (hors coût des grands entiers), donc la complexité est $\Theta(d) = \Theta(\log n)$ tours.

Question 7. Terminaison. La quantité entière $n \geq 0$ décroît strictement à chaque tour (on passe de n à $\lfloor n/16 \rfloor < n$ tant que $n \geq 1$) et reste positive : c'est un *variant de boucle*, donc la boucle termine.

Correction. Notons $c_{d-1} \dots c_1 c_0$ les chiffres produits (poids faible c_0). Par construction, à chaque tour on extrait $c_j = n_j \bmod 16$ puis $n_{j+1} = \lfloor n_j/16 \rfloor$, avec $n_0 = n$. Par la division euclidienne, $n_j = 16 n_{j+1} + c_j$ avec $0 \leq c_j < 16$. Une récurrence immédiate donne $n = \sum_{j=0}^{d-1} c_j 16^j$. Or `hex_vers_entier` évalue par schéma de Horner exactement $\sum_{j=0}^{d-1} c_j 16^j$. Donc `hex_vers_entier(entier_vers_hex(n))` est égal à n . Le cas $n = 0$ est traité à part et renvoie "0", décodé en 0. $\square$

Vérification numérique : $255 \mapsto \text{"ff"}$, $16 \mapsto \text{"10"}$, $7271972807 \mapsto \text{"1b1717fc7"}$ (9 chiffres, et $\lfloor \log_{16} 7271972807 \rfloor + 1 = 9$).

Partie III — Arbres de Merkle

Question 8.

```

1 def parent(a, b):
2     return (31 * a + b) % 1000003
3
4 def racine_merkle(feuilles):
5     niveau = list(feuilles)
6     while len(niveau) > 1:
7         if len(niveau) % 2 == 1:
8             niveau.append(niveau[-1])
9             niveau = [parent(niveau[i], niveau[i+1])
10                        for i in range(0, len(niveau), 2)]
11     return niveau[0]
```

Vérification : niveau 0 = [1, 2, 3, 4] ; `parent(1, 2) = 33`, `parent(3, 4) = 97` ; niveau 1 = [33, 97] ; `parent(33, 97) = 33*31+97 = 1120`. Donc la racine vaut **1120**. (De même `racine_merkle([1, 2, 3]) = 1119` après duplication du 3.)

Question 9. À chaque niveau, si la liste a une taille t , on effectue $\lceil t/2 \rceil$ appels à `parent` et la taille suivante est $\lceil t/2 \rceil$. Partant de $t_0 = n$, on a $t_{k+1} = \lceil t_k/2 \rceil \leq t_k/2 + 1$. Le nombre total d'appels est $\sum_{k \geq 0} \lceil t_k/2 \rceil = \sum_{k \geq 1} t_k \leq \sum_{k \geq 1} (n/2^k + 2)$. La partie géométrique se majore par n , et il y a $O(\log n)$ niveaux, donc le total est $\leq n + O(\log n) = O(n)$. $\square$

Question 10.

```

1 def preuve_inclusion(feuilles, i):
2     niveau = list(feuilles)
3     preuve = []
4     idx = i
5     while len(niveau) > 1:
6         if len(niveau) % 2 == 1:
7             niveau.append(niveau[-1])
8         if idx % 2 == 0:
9             preuve.append((niveau[idx+1], 'R'))
10        else:
11            preuve.append((niveau[idx-1], 'L'))
12        niveau = [parent(niveau[j], niveau[j+1])
```

```

13         for j in range(0, len(niveau), 2)]
14         idx //= 2
15     return preuve
16
17 def verifier(feuille, preuve, racine):
18     acc = feuille
19     for frere, cote in preuve:
20         if cote == 'R':
21             acc = parent(acc, frere)
22         else:
23             acc = parent(frere, acc)
24     return acc == racine

```

Vérification : `preuve_inclusion([1, 2, 3, 4], 0) = [(2, 'R'), (97, 'R')]` ; puis `parent(1, 2) = 33`, `parent(33, 97) = 1120 = racine`.

Question 11. Récurrence sur la hauteur h . Pour $h = 0$ (une seule feuille), la racine *est* la feuille ; une preuve vide n'est acceptée que si `feuille = racine`. Supposons la propriété vraie à la hauteur h et considérons un arbre de hauteur $h + 1$. La vérification recalcule d'abord $r' = \text{parent}(\text{feuille}, \text{frère})$ (ou l'inverse), puis vérifie que r' , muni du reste de la preuve, mène à la racine : c'est une preuve d'inclusion de r' dans un arbre de hauteur h , valide par hypothèse de récurrence, donc r' est bien un nœud interne du niveau h . Comme `parent` est résistante aux collisions, l'unique antécédent connu de r' est le couple `(feuille, frère)` réellement présent ; la feuille figure donc parmi celles ayant produit la racine. $\square$

La preuve contient un couple par niveau, soit une taille $O(\log n)$: c'est l'intérêt majeur des arbres de Merkle (certificat logarithmique).

Partie IV — Sélection aléatoire d'un validateur

Question 12. Le premier succès arrive au k -ième créneau si les $k - 1$ premiers sont des échecs (probabilité $(1 - p)^{k-1}$) suivis d'un succès (probabilité p) :

$$\mathbb{P}(N = k) = (1 - p)^{k-1} p, \quad k \geq 1.$$

C'est la *loi géométrique* de paramètre p , d'espérance $\mathbb{E}[N] = \sum_{k \geq 1} k(1 - p)^{k-1} p = \frac{1}{p}$.

Question 13. $\mathbb{P}(N > k)$ est la probabilité de k échecs consécutifs, soit $(1 - p)^k$. Comme $0 < 1 - p < 1$, $(1 - p)^k \rightarrow 0$ quand $k \rightarrow +\infty$, donc $\mathbb{P}(N < +\infty) = 1 - \lim_k \mathbb{P}(N > k) = 1$: terminaison presque sûre. $\square$

Question 14. Pour $p = 1/4$: $\mathbb{E}[N] = 4$, et $\mathbb{P}(N \leq 4) = 1 - (3/4)^4 = 1 - \frac{81}{256} = \frac{175}{256} \approx 0,684$.

Partie V — Partition équilibrée par programmation dynamique

Question 15. Si T est impaire, aucune part entière ne peut valoir $T/2$ (qui n'est pas entier) ; les deux parts, de sommes entières égales, donneraient T pair. Donc la réponse est *non*. $\square$

Question 16. Un *certificat* est la liste des indices d'un sous-ensemble candidat. La vérification calcule sa somme et teste l'égalité à C , en temps $O(n)$ (polynomial). Le problème est donc dans **NP**. (Il s'agit du problème SUBSET-SUM, NP-complet.)

Question 17. Un sous-ensemble des i premiers éléments de somme c ou bien n'utilise pas le i -ième élément (donc c'est un sous-ensemble des $i - 1$ premiers de somme c), ou bien l'utilise (donc c'est un sous-ensemble des $i - 1$ premiers de somme $c - S[i - 1]$, licite si $c \geq S[i - 1]$) :

$$A[i][c] = A[i - 1][c] \vee (c \geq S[i - 1] \wedge A[i - 1][c - S[i - 1]]),$$

avec l'initialisation $A[0][0] = \text{vrai}$ et $A[0][c] = \text{faux}$ pour $c \geq 1$.

Question 18.

```

1 def peut_partitionner(S):
2     T = sum(S)
3     if T % 2 == 1:
4         return False
5     C = T // 2
6     dp = [False] * (C + 1)
7     dp[0] = True
8     for x in S: # traite un element a la fois
9         for c in range(C, x - 1, -1): # parcours DECROISSANT
10             if dp[c - x]:
11                 dp[c] = True
12     return dp[C]

```

Le parcours *décroissant* de c garantit que $dp[c - x]$ se réfère encore à l'état *avant* l'introduction de x : on n'utilise donc jamais deux fois le même élément. Complexité : $O(nC)$ en temps et $O(C)$ en espace. Comme C dépend de la *valeur* des données (et non seulement de leur nombre), la taille de C est exponentielle en le nombre de bits de codage : l'algorithme est dit *pseudo-polynomial*.

Question 19. Invariant. Après le traitement des i premiers éléments, pour tout $0 \leq c \leq C$, $dp[c]$ est vrai si et seulement s'il existe un sous-ensemble des i premiers éléments de somme c (c'est-à-dire $dp[c] = A[i][c]$).

Initialisation ($i = 0$) : seul $dp[0]$ est vrai, ce qui correspond au sous-ensemble vide ; conforme à $A[0]$. *Hérédité* : supposons l'invariant vrai après $i - 1$ éléments et traitons $x = S[i - 1]$. Pour chaque c de C à x , on met $dp[c]$ à vrai lorsque $dp[c - x]$ est vrai. Le parcours décroissant assure que $dp[c - x]$ vaut encore $A[i - 1][c - x]$; combiné à la valeur préexistante $A[i - 1][c]$, on obtient exactement $A[i][c] = A[i - 1][c] \vee (c \geq x \wedge A[i - 1][c - x])$. L'invariant est donc vrai après i éléments.

Conclusion : après les n éléments, $dp[C] = A[n][C]$, qui est vrai ssi une partition équilibrée existe.

Terminaison : les deux boucles **for** sont finies. $\square$

Vérifications numériques : `peut_partitionner([1, 1, 5, 7]) = True` (part $[1, 1, 5]$ de somme 7), `peut_partitionner([1, 1, 5, 8]) = False` (somme 15 impaire). Sur 2000 jeux aléatoires, `peut_partitionner` coïncide avec une recherche exhaustive.